

Sistemas Operacionais

Paginação em disco

Professor Romário da Silva Borges

Armazenamento externo

A memória RAM sempre foi um recurso escasso em sistemas de computação. Por isso, seu uso deve ser gerenciado de forma eficiente, para que todos os processos possam ter memória suficiente para operar. Além disso, a crescente manipulação de informações multimídia (imagens, áudio, vídeo) contribui para esse problema, uma vez que essas informações são geralmente volumosas e seu tratamento exige grandes quantidades de memória livre.

Esta aula apresenta técnicas usadas para usar um dispositivo de armazenamento secundário como extensão da memória RAM, de forma transparente para as aplicações.

Estendendo a memória RAM

Os mecanismos de memória virtual suportados pelo hardware, permitem usar dispositivos de armazenamento secundário como extensão da memória RAM. Com isso, partes ociosas da memória podem ser transferidas para um disco, liberando a memória RAM para outros usos. Caso algum processo tente acessar esse conteúdo posteriormente, ele deverá ser trazido de volta à memória. A transferência dos dados entre memória e disco é feita pelo sistema operacional, de forma transparente para os processos.

Existem diversas técnicas para usar um espaço de armazenamento secundário como extensão da memória RAM, com ou sem o auxílio do hardware.

Overlays

O programador organiza seu programa em módulos que serão carregados em uma mesma região de memória em momentos distintos. Esses módulos, chamados de overlays, são gerenciados através de uma biblioteca específica. Por exemplo, o código de um compilador pode separar o analisador léxico, o analisador sintático e o gerador de código em overlays, que serão ativados em momentos distintos. Esta abordagem, popular em alguns ambientes de desenvolvimento nos anos 1970-90, como o Turbo Pascal da empresa Borland, é raramente usada hoje em dia, pois exige maior esforço por parte do programador.

Swapping

Consiste em mover um processo ocioso da memória RAM para um disco (swap-out), liberando a memória para outros processos. Mais tarde, quando esse processo for acordado (entrar na fila de prontos do escalonador), ele é carregado de volta na memória (swap-in). A técnica de swapping foi muito usada até os anos 1990, mas hoje é pouco empregada em sistemas operacionais de uso geral.

Paginação

Consiste em mover páginas individuais, conjuntos de páginas ou mesmo segmentos da memória para o disco (page-out). Se o processo tentar acessar uma dessas páginas mais tarde, a MMU gera uma interrupção de falta de página e o núcleo do SO recarrega a página faltante na memória (page-in). Esta é a técnica mais usada nos sistemas operacionais atuais, por sua flexibilidade, rapidez e eficiência.

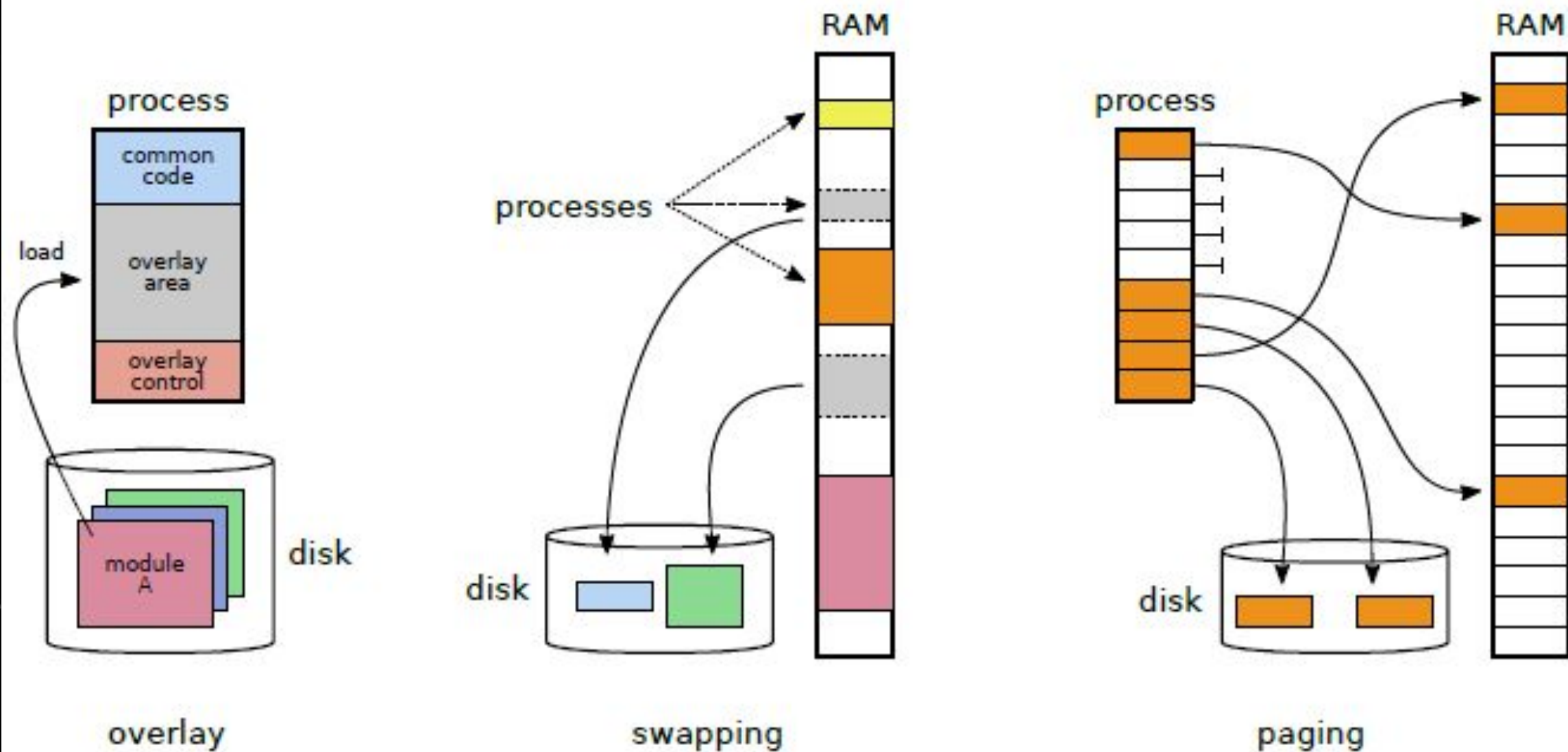


Figura 17.1: Abordagens de extensão da memória em disco.

Paginação em disco

Mecanismo

A ideia central da paginação em disco consiste em transferir páginas ociosas da memória RAM para uma área em disco, liberando memória para outras páginas.

A transferência de páginas entre a memória e o disco é realizada pelo núcleo do sistema operacional. As páginas a retirar da memória são escolhidas por ele, de acordo com algoritmos de substituição de páginas. Quando um processo tentar acessar uma página que está em disco, o núcleo é alertado pela MMU e traz a página de volta à memória para poder ser acessada.

Para cada página transferida para o disco, a tabela de páginas do processo é ajustada: o flag de presença da página em RAM é desligado e a posição da página no disco é registrada, ao invés do quadro.

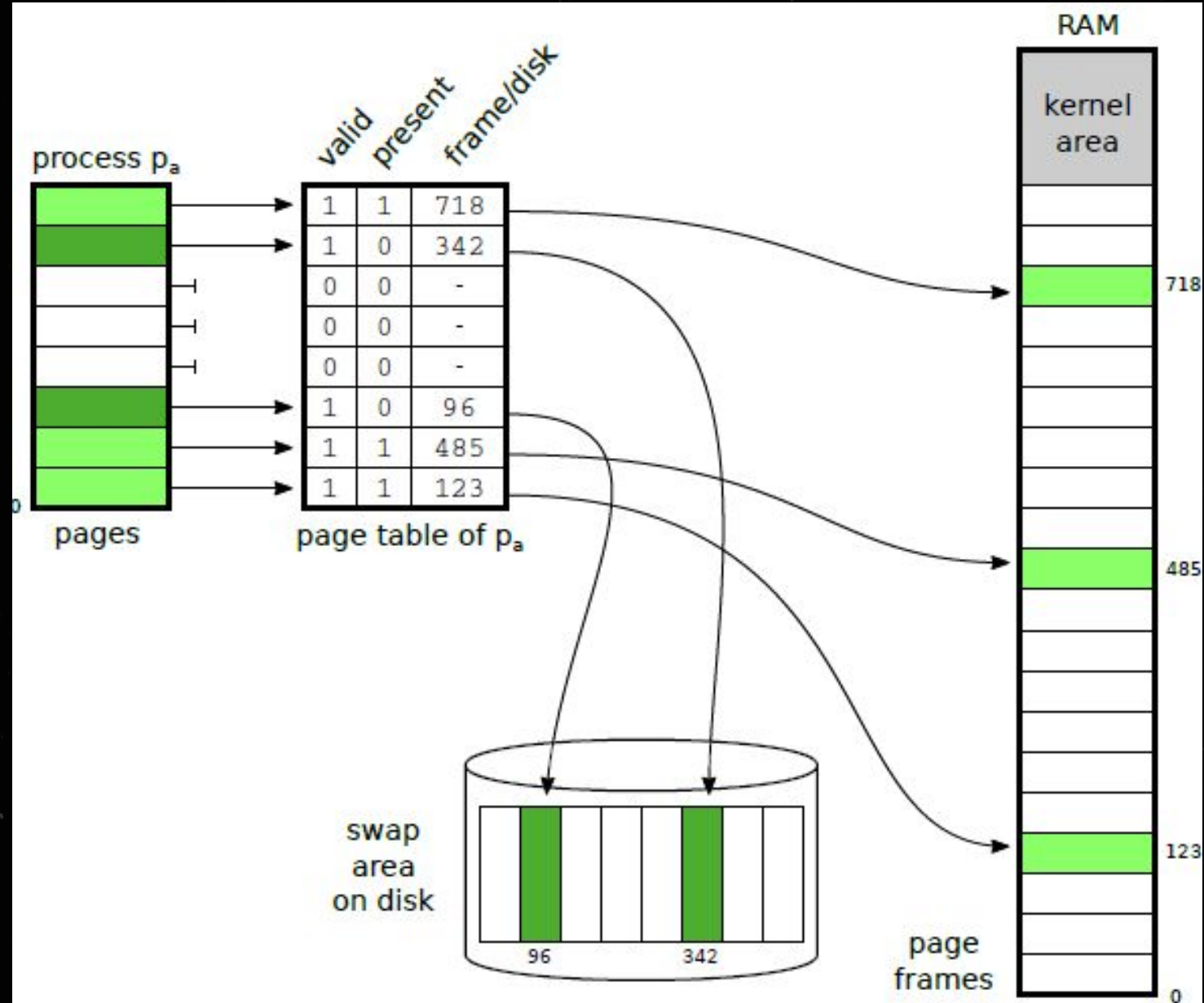
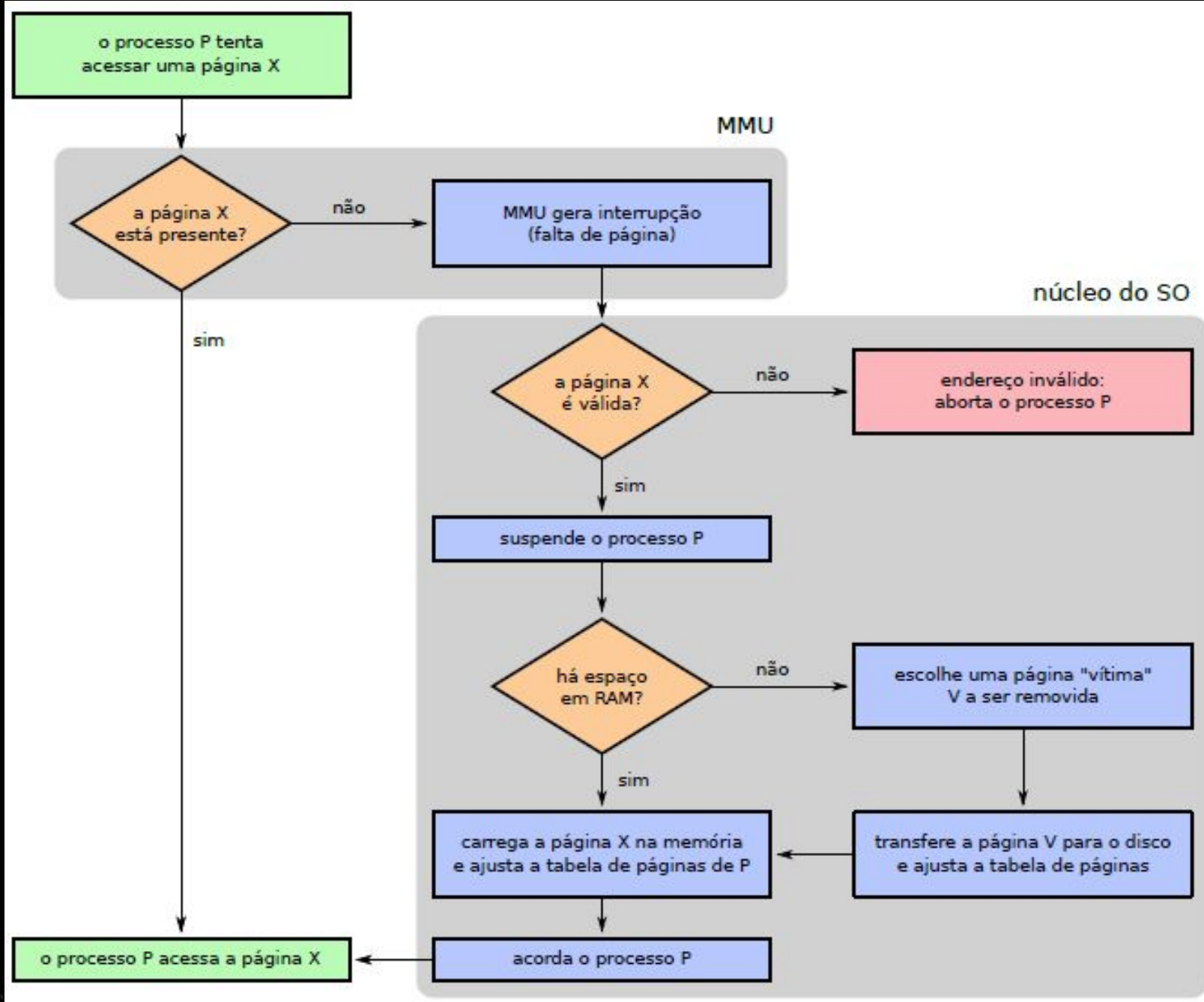


Figura 17.2: A paginação em disco.

Page fault

Quando um processo acessa uma página, a MMU verifica se a mesma está presente na memória RAM; em caso positivo, faz o acesso ao endereço físico correspondente. Caso contrário, a MMU gera uma interrupção de **falta de página (*page fault*)** que desvia a execução para o sistema operacional.





Eficiência

Paginação em disco: permite usar o disco como uma extensão da memória RAM, de forma transparente para os processos.

Principal problema: o acesso ao disco é muito mais lento que o acesso à memória RAM.

Diferença de velocidade: um disco rígido típico pode ter um tempo de acesso cerca de 100.000 vezes maior que a RAM.

Como escolher a página a ser removida?

RAM cheia: podem ser necessários dois acessos ao disco: um para retirar uma página da RAM e outro para carregar a página necessária.

Impacto no desempenho: muitas faltas de página provocam muitos acessos ao disco, aumentando significativamente o tempo médio de acesso à memória.

A escolha das páginas a remover da memória: caso sejam removidas páginas usadas com muita frequência, estas serão provavelmente acessadas pouco tempo após sua remoção, gerando mais faltas de página.

Critérios de seleção

Idade da página: páginas que estão há muito tempo na memória podem ser candidatas à remoção por possivelmente serem pouco utilizadas.

Frequência de acessos: páginas acessadas com muita frequência recentemente tendem a continuar sendo utilizadas.

Data do último acesso: páginas que não são acessadas há mais tempo têm maior chance de permanecer sem uso.

Critérios de seleção

Prioridade do processo: páginas de processos de alta prioridade ou de tempo real devem preferencialmente permanecer na RAM para evitar atrasos.

Conteúdo da página: páginas de código são mais fáceis de recuperar do disco, enquanto páginas de dados modificadas precisam ser salvas antes da remoção.

Páginas especiais: páginas relacionadas à entrada/saída ou contendo dados sensíveis podem precisar permanecer na RAM por questões de funcionamento ou segurança.

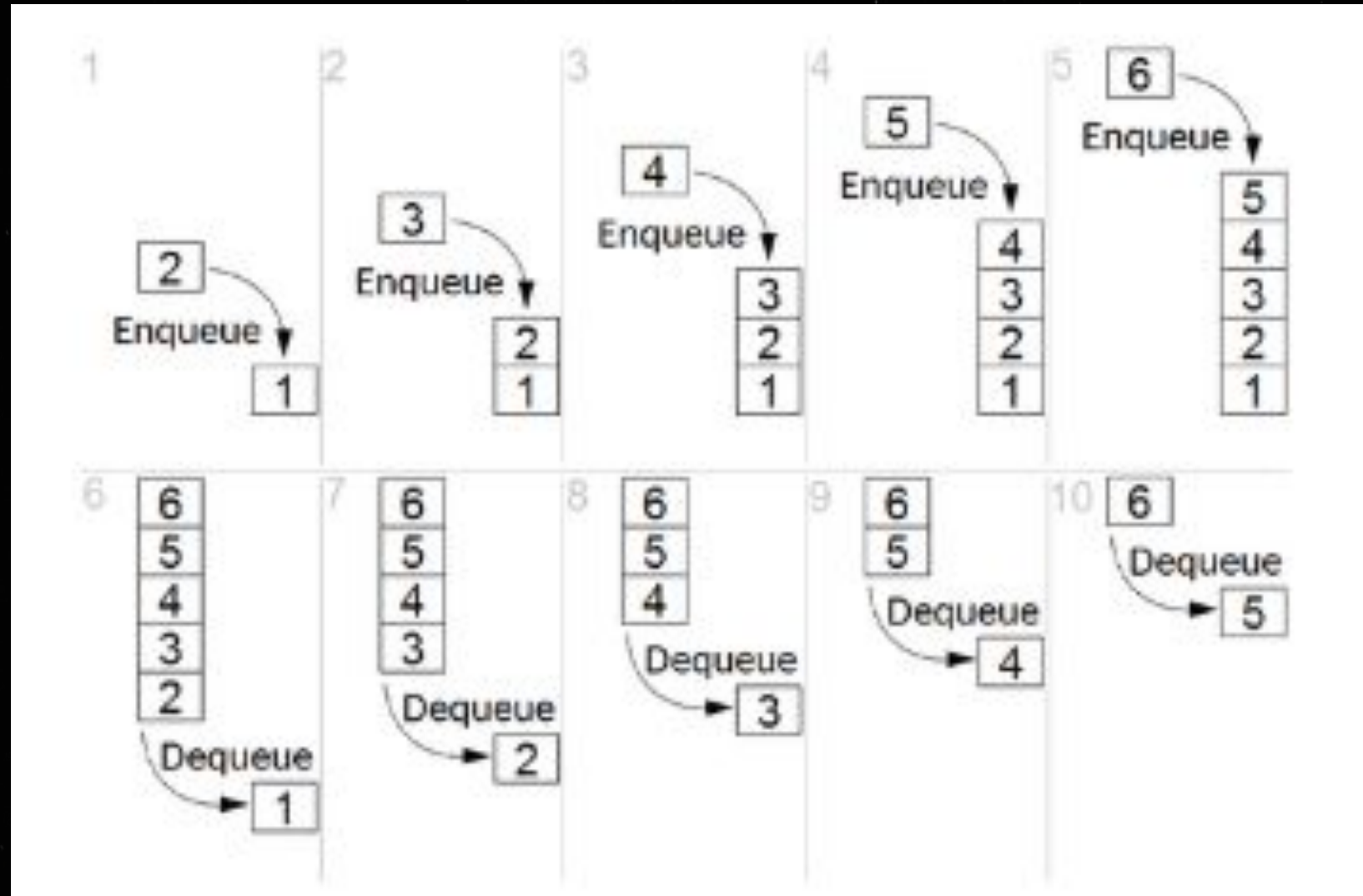
Algoritmos de Substituição de páginas

Para possibilitar a comparação dos algoritmos de substituição de páginas apresentados na sequência, será usada a seguinte cadeia de referências fictícia, com 20 acessos a 6 páginas distintas:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

Deve-se observar que acessos consecutivos a uma mesma página não são relevantes para a análise dos algoritmos, porque somente o primeiro acesso em cada grupo de acessos consecutivos à mesma página provoca uma falta de página. (OBS: as imagens a seguir apresentam parte das imagens. Consulte o livro de Maziero para a versão completa.)

FIFO



FIFO

t	página acessada	quadros			falta de página?	ação realizada
		q_0	q_1	q_2		
0						situação inicial, quadros vazios
1	7	7			✓	p_7 é carregada em q_0
2	0	7	0		✓	p_0 é carregada em q_1
3	1	7	0	1	✓	p_1 é carregada em q_2
4	2	2	0	1	✓	p_2 substitui p_7 (carregada em $t = 1$)
5	0	2	0	1		p_0 já está na memória
6	3	2	3	1	✓	p_3 substitui p_0
7	0	2	3	0	✓	p_0 substitui p_1
8	4	4	3	0	✓	p_4 substitui p_2
9	2	4	2	0	✓	p_2 substitui p_3
10	3	4	2	3	✓	p_3 substitui p_0
11	0	0	2	3	✓	p_0 substitui p_4
12	3	0	2	3		p_3 já está na memória

Ótimo

Idealmente, a melhor página a remover da memória em um dado instante é aquela que ficará mais tempo sem ser usada pelos processos. Esta ideia simples define o algoritmo ótimo (OPT).

Entretanto, como o comportamento futuro dos processos não pode ser previsto com precisão, este algoritmo não é implementável. Mesmo assim ele é importante, porque define um limite mínimo conceitual: se, para uma dada cadeia de referências, o algoritmo ótimo gera N faltas de página, nenhum outro algoritmo irá gerar menos de N faltas de página ao tratar essa mesma cadeia.

Assim, seu resultado serve como parâmetro para a avaliação dos demais algoritmos.

Ótimo

t	página acessada	quadros			falta de página?	ação realizada
		q_0	q_1	q_2		
0						situação inicial, quadros vazios
1	7	7			✓	p_7 é carregada em q_0
2	0	7	0		✓	p_0 é carregada em q_1
3	1	7	0	1	✓	p_1 é carregada em q_2
4	2	2	0	1	✓	p_2 substitui p_7 (que só será acessada em $t = 18$)
5	0	2	0	1		p_0 já está na memória
6	3	2	0	3	✓	p_3 substitui p_1
7	0	2	0	3		p_0 já está na memória
8	4	2	4	3	✓	p_4 substitui p_0
9	2	2	4	3		p_2 já está na memória
10	3	2	4	3		p_3 já está na memória
11	0	2	0	3	✓	p_0 substitui p_4
12	3	2	0	3		p_3 já está na memória
13	2	2	0	3		p_2 já está na memória

LRU (*Least Recently Used*, menos recentemente usado)

Put(7)	7		
Put(10)	10	7	
Put(8)	8	10	7
Put(5)	5	8	10

LRU (*Least Recently Used*, menos recentemente usado)

t	página	quadros			falta de página?	ação realizada
	acessada	q_0	q_1	q_2		
0						situação inicial, quadros vazios
1	7	7			✓	p_7 é carregada em q_0
2	0	7	0		✓	p_0 é carregada em q_1
3	1	7	0	1	✓	p_1 é carregada em q_2
4	2	2	0	1	✓	p_2 substitui p_7 (há mais tempo sem acesso)
5	0	2	0	1		p_0 já está na memória
6	3	2	0	3	✓	p_3 substitui p_1
7	0	2	0	3		p_0 já está na memória
8	4	4	0	3	✓	p_4 substitui p_2
9	2	4	0	2	✓	p_2 substitui p_3
10	3	4	3	2	✓	p_3 substitui p_0
11	0	0	3	2	✓	p_0 substitui p_4
12	3	0	3	2		p_3 já está na memória
13	2	0	3	2		p_2 já está na memória

Algoritmo RANDOM

Um algoritmo interessante consiste em escolher aleatoriamente as páginas a retirar da memória. O algoritmo aleatório pode ser útil em situações onde as abordagens LRU e FIFO têm desempenho ruim, como os padrões de acesso fortemente sequenciais discutidos na seção anterior.

De fato, alguns sistemas operacionais mais antigos usavam essa abordagem em situações onde os demais algoritmos funcionavam mal.

Comparação entre algoritmos

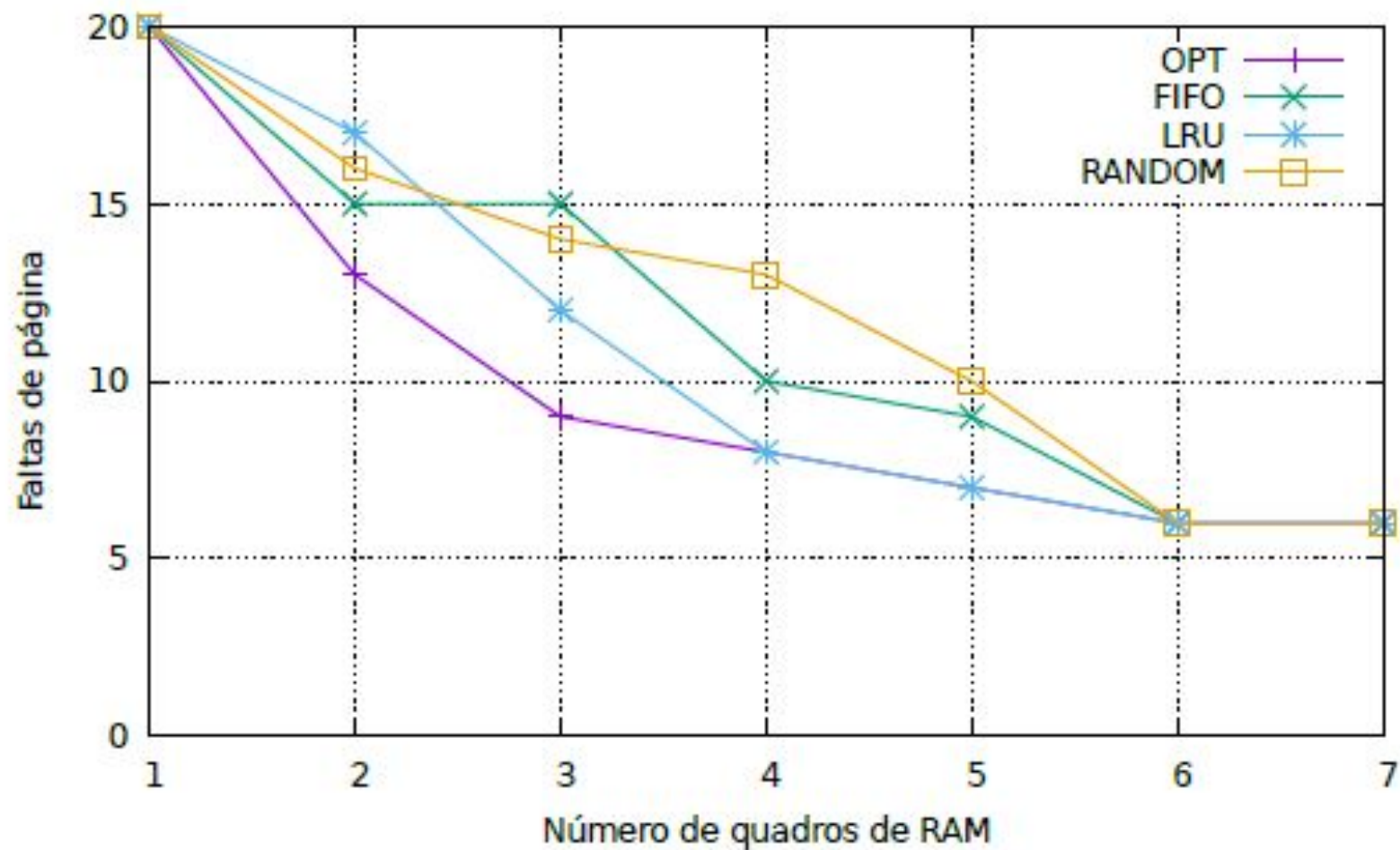
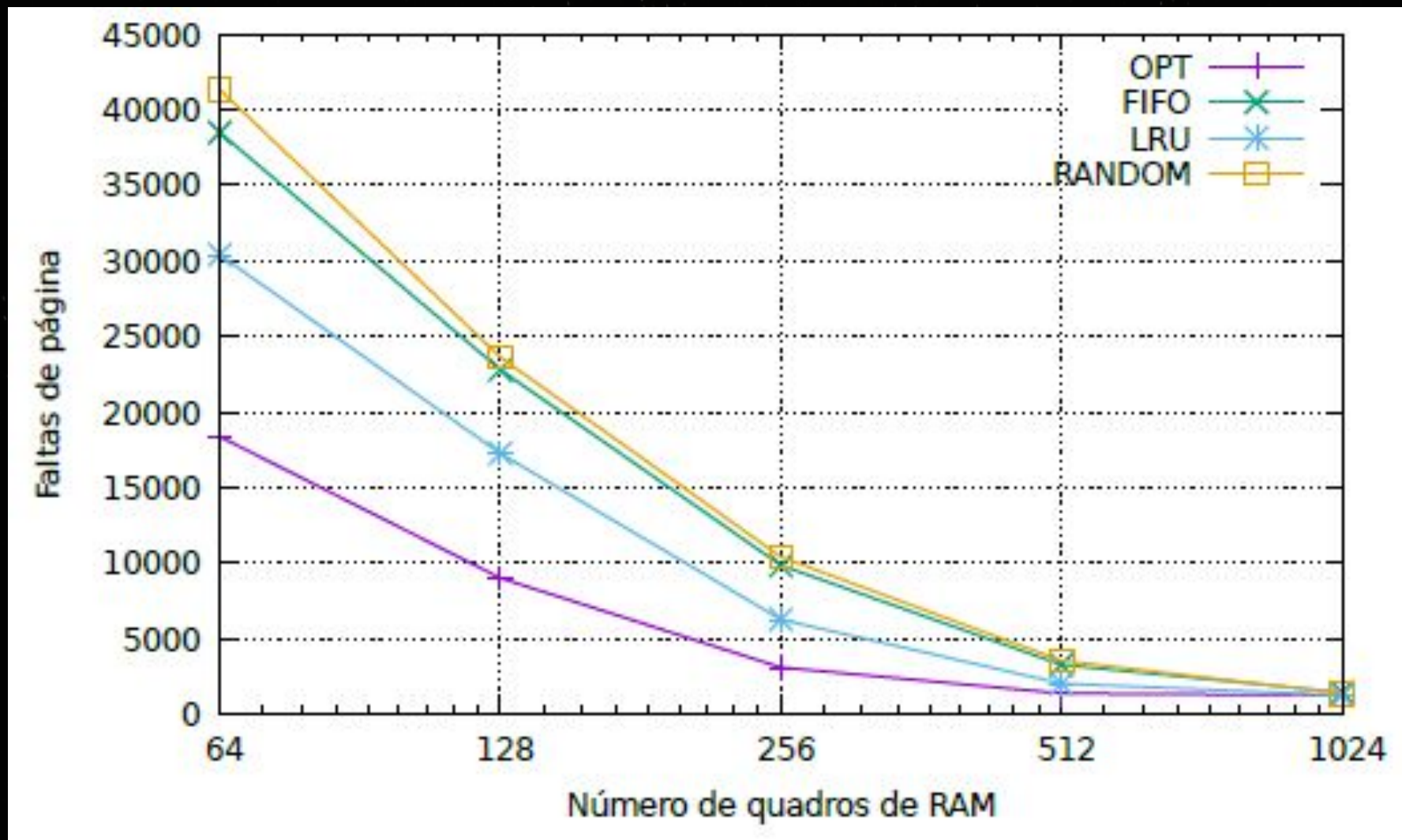


Figura 17.4: Comparação dos algoritmos de substituição de páginas.

Comparação entre algoritmos



Outros algoritmos

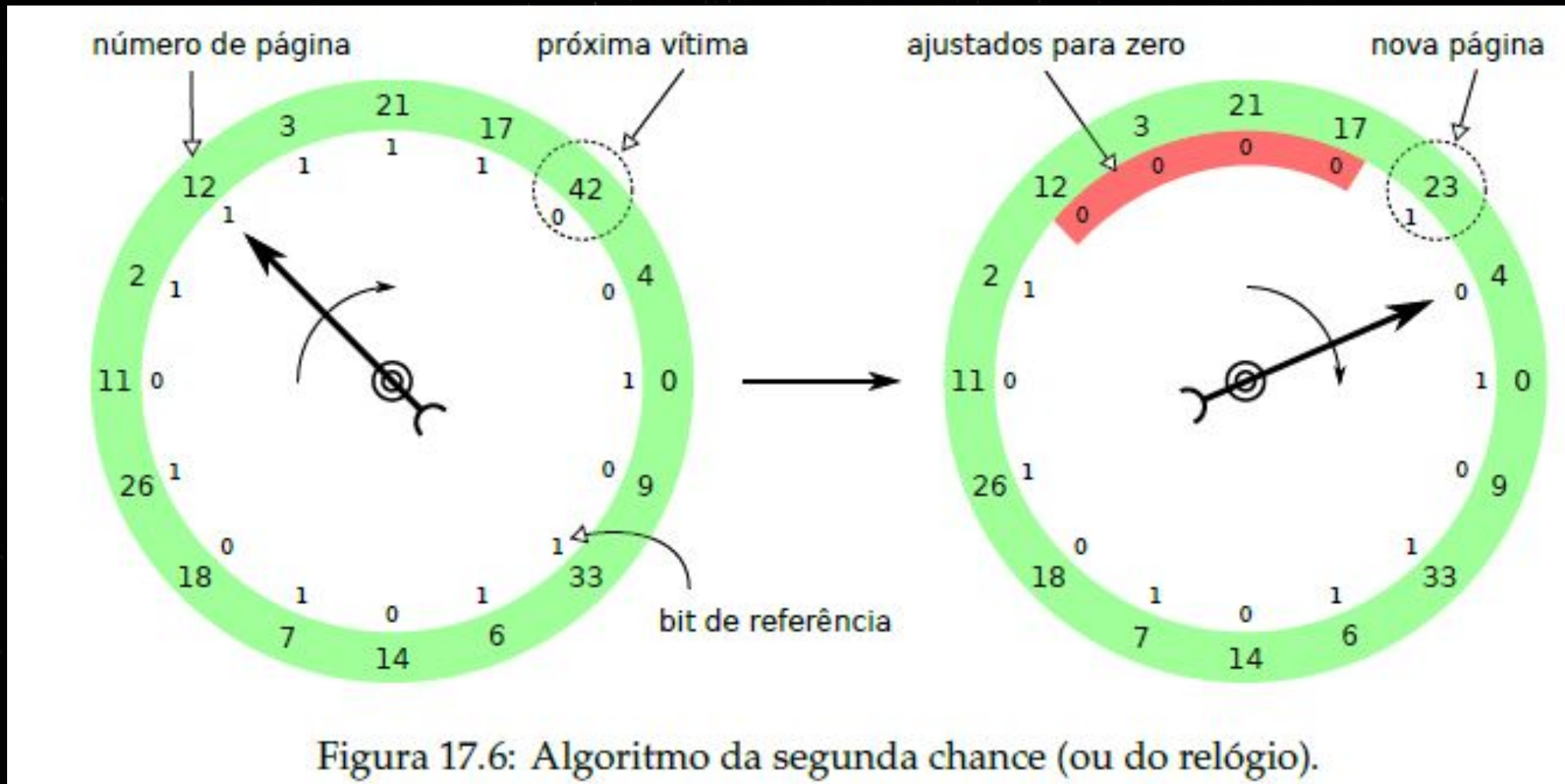
Embora possa ser implementado, o algoritmo LRU completo é pouco usado na prática, porque sua implementação exigiria registrar as datas de acesso às páginas a cada leitura ou escrita na memória, o que é difícil de implementar de forma eficiente em software e com custo proibitivo para implementar em hardware.

Portanto, a maioria dos sistemas operacionais reais implementa algoritmos baseados em aproximações do LRU.

Algoritmo Segunda Chance

É uma simples modificação do algoritmo FIFO. Utiliza a Fila, mas evita eliminar uma página recentemente utilizada. Mescla a ideia do FIFO, mas inserindo informações de popularidade.

Algoritmo Segunda Chance



Algoritmo NRU (*Not Recently Used*, ou não usada recentemente)

O algoritmo da segunda chance leva em conta somente o bit de referência de cada página ao escolher as vítimas para substituição. O algoritmo NRU (*Not Recently Used*, ou não usada recentemente) melhora essa escolha, ao considerar também o bit de modificação.

O algoritmo NRU consiste simplesmente em tentar substituir primeiro páginas do nível 0; caso não encontre, procura candidatas no nível 1 e assim sucessivamente. Pode ser necessário percorrer várias vezes a lista circular até encontrar uma página adequada para substituição.

Algoritmo NRU

R

Bit de Referência (R)

Indica se a página foi acessada (referenciada) recentemente.

- R = 1 → foi acessada
- R = 0 → não foi acessada

M

Bit de Modificação (M)

Indica se o conteúdo da página foi alterado desde que foi carregada na memória.

- M = 1 → foi modificada (suja)
- M = 0 → não foi modificada (limpa)

Classe	R	M	Descrição	Importância	Ação na substituição
00	0	0	Não referenciada recentemente e não modificada.	Melhor candidata	Pode ser simplesmente retirada da memória (não precisa gravar).
01	0	1	Não referenciada recentemente, mas modificada.	Candidata razoável	Deverá ser gravada na área de troca (swap) antes de ser substituída.
10	1	0	Referenciada recentemente e não modificada.	Não é uma boa escolha	Provavelmente será referenciada novamente em breve.
11	1	1	Referenciada recentemente e modificada.	Pior escolha	Terá de ser gravada na área de troca e provavelmente será necessária novamente em breve.

Algoritmo NRU

Considere as páginas em memória com seus bits R e M:

Página	R	M	Classe
P1	0	0	00
P2	0	1	01
P3	1	0	10
P4	1	1	11
P5	0	0	00
P6	1	0	10
P7	0	1	01
P8	1	1	11



Ordem de seleção no NRU
(da melhor para a pior)

1

Classe 00
(ex.: P1, P5)

2

Classe 01
(ex.: P2, P7)

3

Classe 10
(ex.: P3, P6)

4

Classe 11
(ex.: P4, P8)

Algoritmo do envelhecimento

O contador construído por este algoritmo constitui uma aproximação razoável do algoritmo LRU: páginas menos acessadas “envelhecerão”, ficando com contadores menores, enquanto páginas mais acessadas permanecerão “jovens”, com contadores maiores.

$$\begin{array}{l}
 p_1 \\
 p_2 \\
 p_3 \\
 p_4
 \end{array}
 \left[\begin{array}{c}
 R \\
 \boxed{0} \\
 \boxed{1} \\
 \boxed{0} \\
 \boxed{1}
 \end{array} \right]
 \text{ com }
 \left[\begin{array}{cc}
 \textit{contadores} & \\
 \boxed{0000 \ 0011} & (3) \\
 \boxed{0011 \ 1101} & (61) \\
 \boxed{1010 \ 1000} & (168) \\
 \boxed{1110 \ 0011} & (227)
 \end{array} \right]
 \Rightarrow
 \left[\begin{array}{c}
 R \\
 \boxed{0} \\
 \boxed{0} \\
 \boxed{0} \\
 \boxed{0}
 \end{array} \right]
 \text{ e }
 \left[\begin{array}{cc}
 \textit{contadores} & \\
 \boxed{0000 \ 0001} & (1) \\
 \boxed{1001 \ 1110} & (158) \\
 \boxed{0101 \ 0100} & (84) \\
 \boxed{1111 \ 0001} & (241)
 \end{array} \right]$$

Anomalia de Belady

Espera-se que, quanto mais memória física um sistema possua, menos faltas de página ocorram. Todavia, esse comportamento intuitivo não se verifica em todos os algoritmos de substituição de páginas. Alguns algoritmos, como o FIFO, podem apresentar um comportamento estranho: ao aumentar o número de quadros de memória, o número de faltas de página geradas pelo algoritmo aumenta, ao invés de diminuir.

Esse comportamento atípico de alguns algoritmos foi estudado pelo matemático húngaro Laslo Belady nos anos 60, sendo por isso denominado anomalia de Belady.

Anomalia de Belady

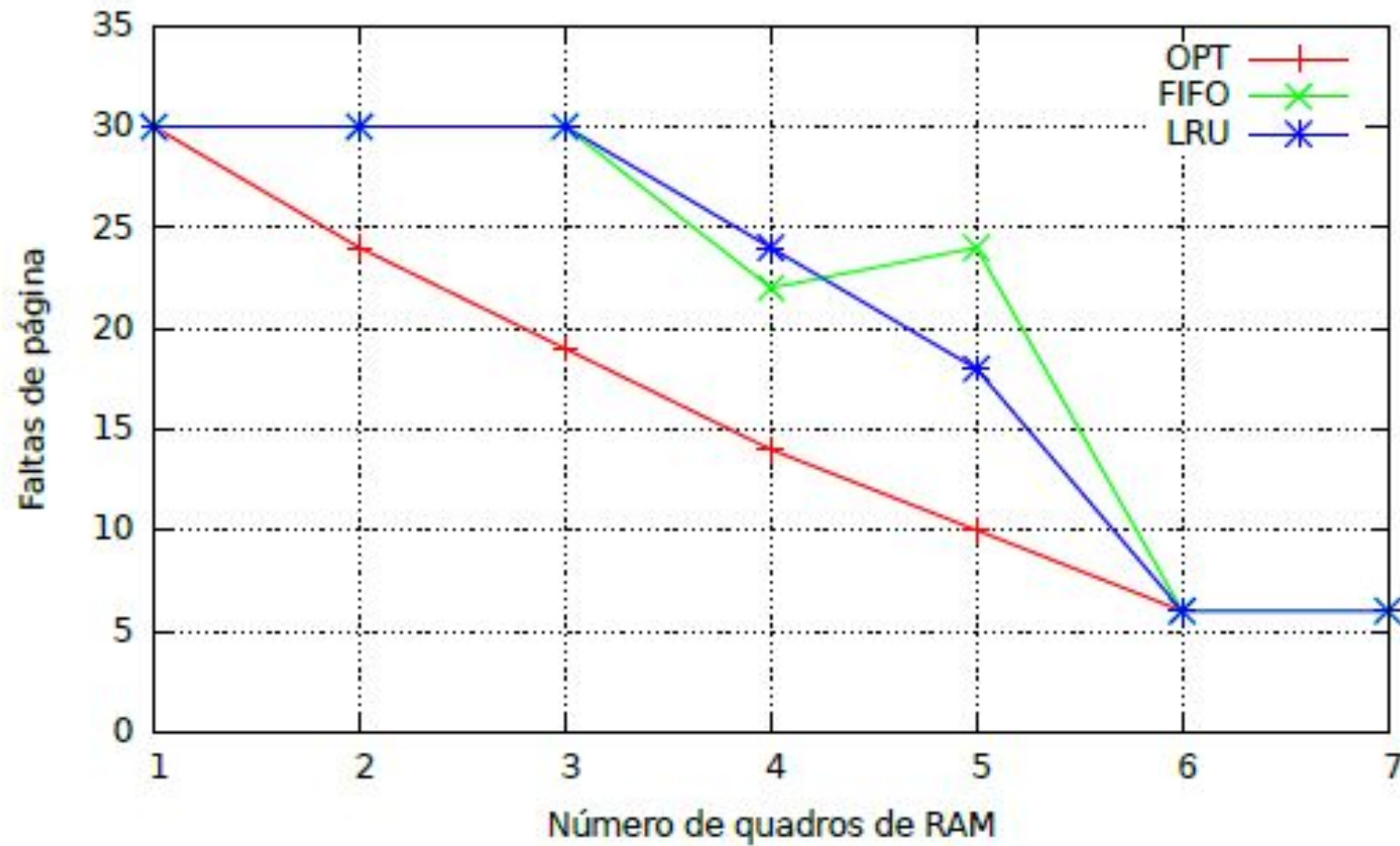


Figura 17.8: A anomalia de Belady.

Anomalia de Belady

Estudos demonstraram que uma família de algoritmos denominada algoritmos de pilha (à qual pertencem os algoritmos OPT e LRU, entre outros) não apresenta a anomalia de Belady.

Logo, não basta apenas ter RAM disponível. Anomalia de Belady demonstra que aumentar a memória disponível nem sempre reduz as faltas de página; isso também depende do algoritmo de substituição utilizado.

Thrashing

No thrashing, a memória RAM não é suficiente para todos os processos ativos, portanto muitos processos não conseguem ter seus conjuntos de trabalho totalmente carregados na memória. Cada vez que um processo recebe o processador, executa algumas instruções, gera uma falta de página e volta ao estado suspenso, até que a página faltante seja trazida de volta à RAM. Todavia, para trazer essa página à RAM será necessário abrir espaço na memória, transferindo algumas páginas (de outros processos) para o disco. Quanto mais processos estiverem nessa situação, maior será a atividade de paginação e maior o número de processos no estado suspenso, aguardando páginas.

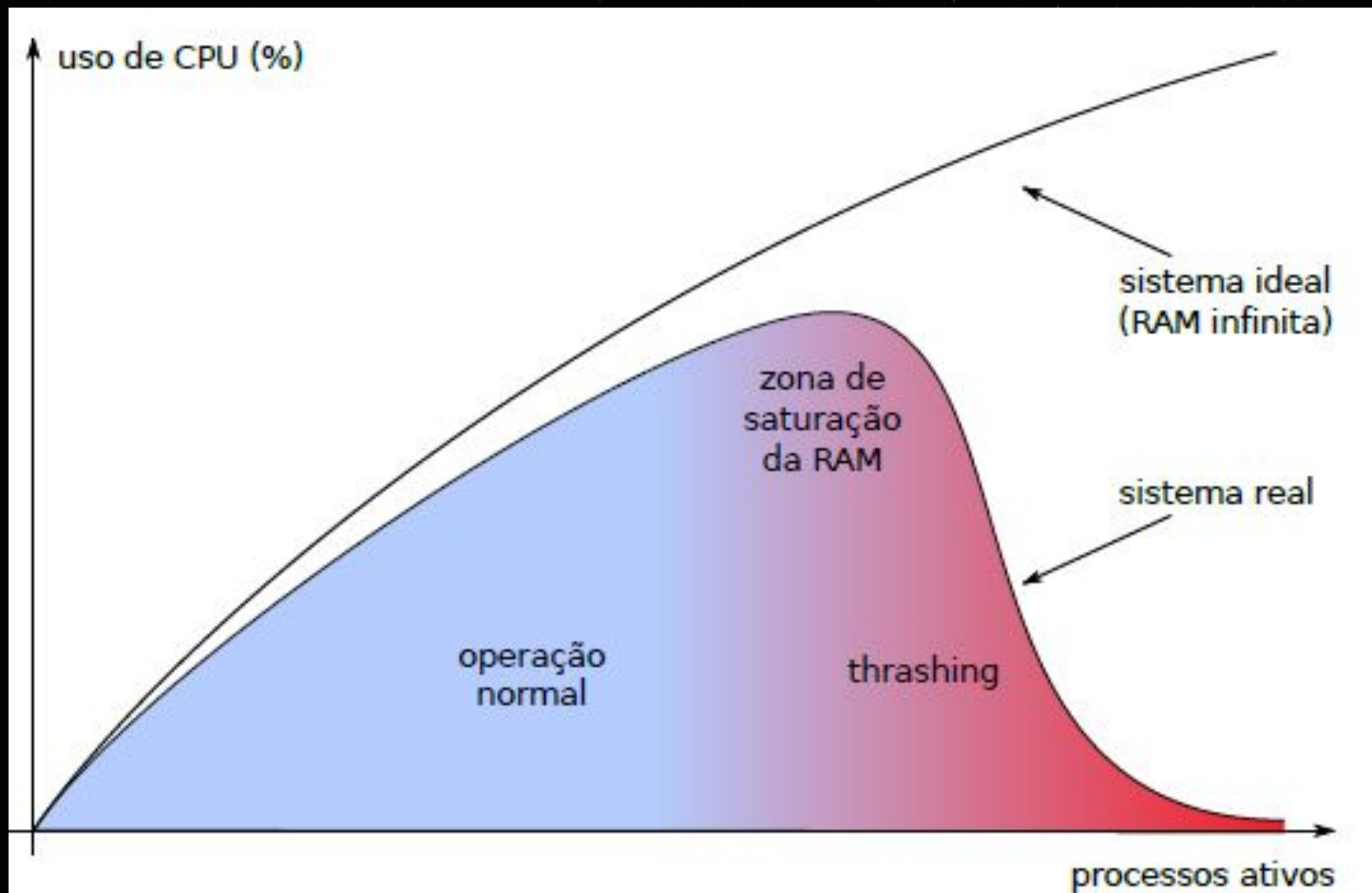


Figura 17.9: Comportamento de um sistema com *thrashing*.

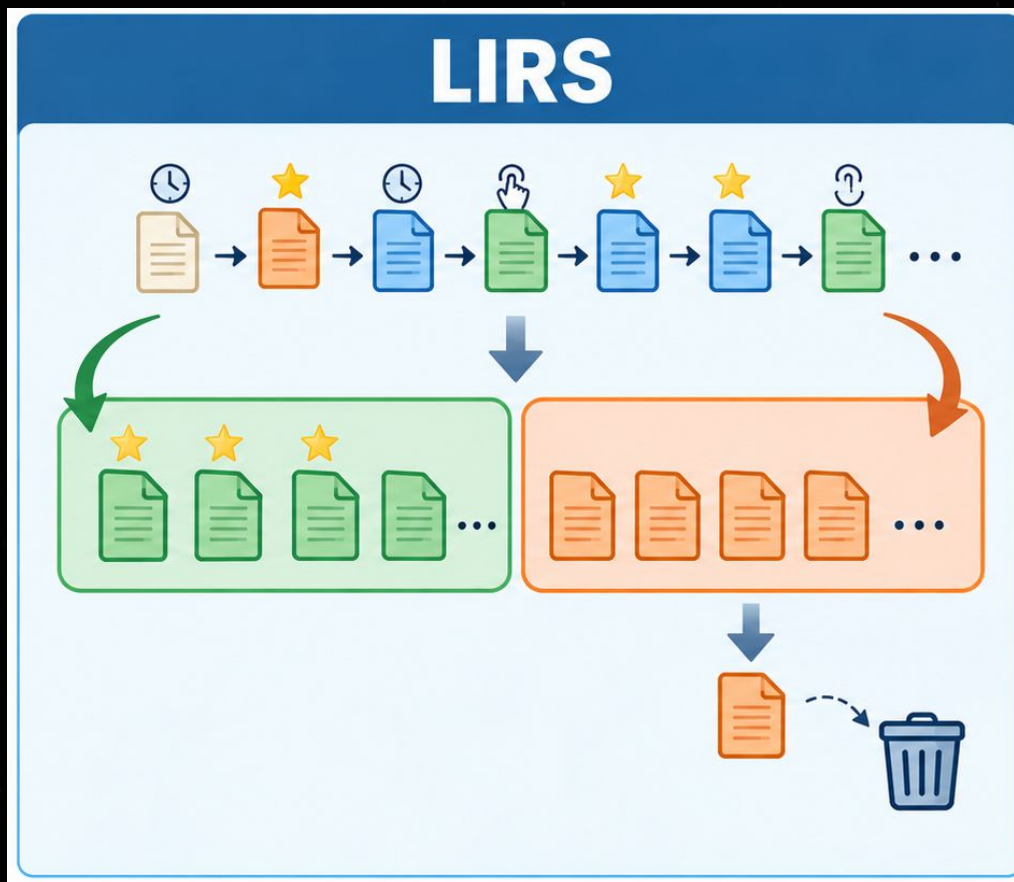
Logo, Thrashing acontece quando o sistema passa a gastar mais tempo tratando faltas de página e movimentando páginas entre RAM e armazenamento do que executando efetivamente os processos.

Algumas soluções atuais

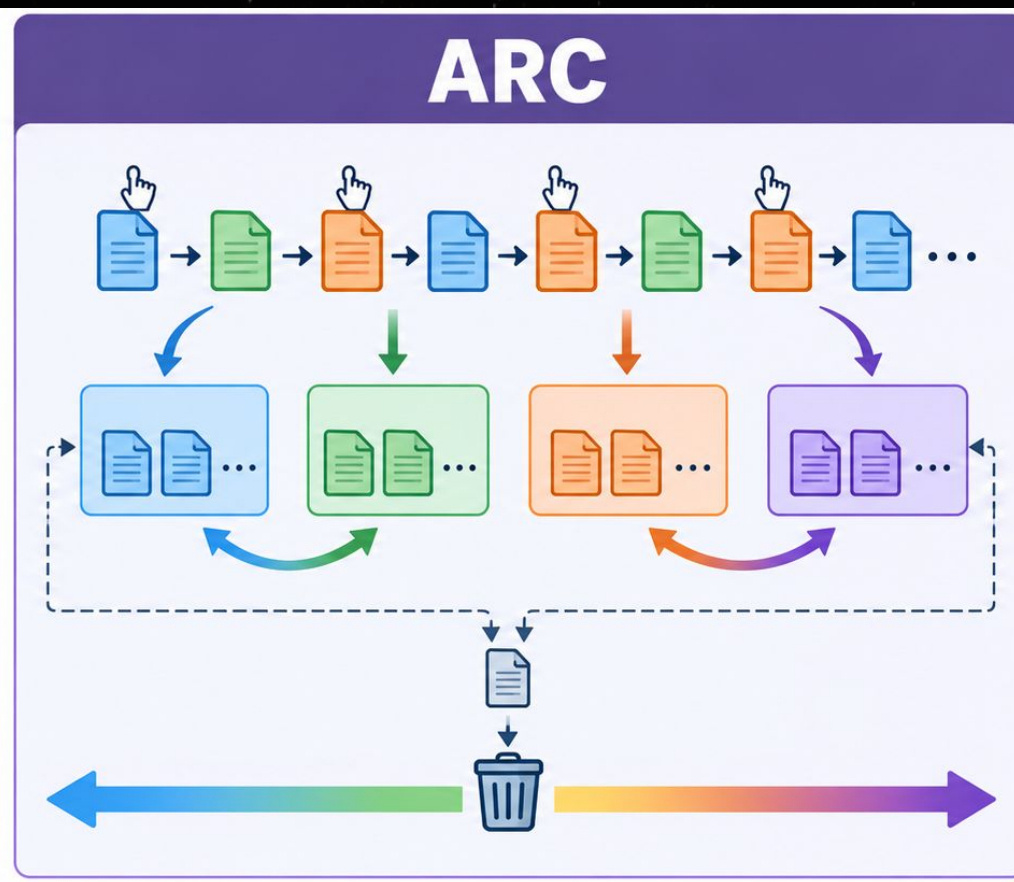
LIRS (Low Inter-reference Recency Set): considera a frequência/recência com que as páginas são reutilizadas, buscando superar limitações do LRU.

ARC (Adaptive Replacement Cache): combina informações de recência e frequência e adapta seu comportamento conforme o padrão de acesso.

Multi-Gen LRU (MGLRU): abordagem mais recente usada no Linux para organizar páginas em gerações conforme seu histórico de acesso, buscando melhorar as decisões de recuperação de memória.



"Qual é o intervalo entre os acessos dessa página?"



"Devo preservar aquilo que foi usado recentemente ou aquilo que é reutilizado frequentemente?"

Multi-Gen LRU (MGLRU)

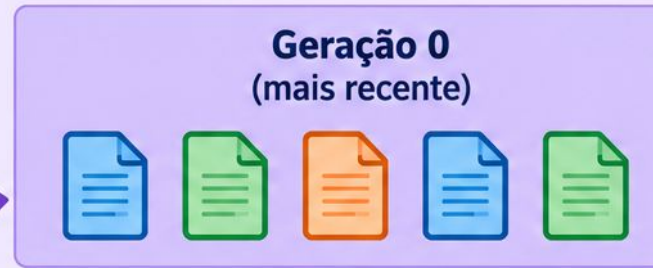


Usado no Linux

1 Acessos às páginas

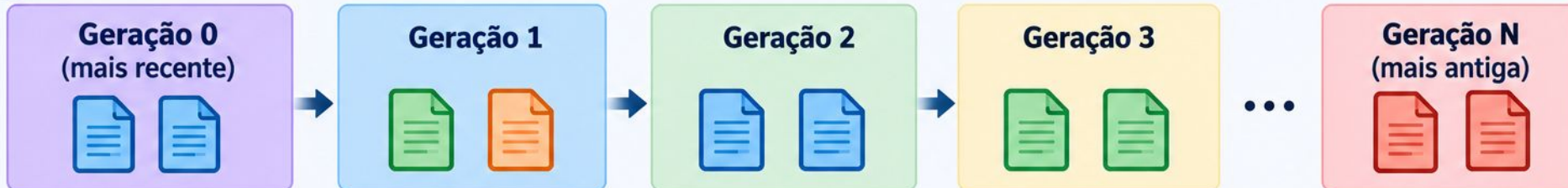


2 Páginas são colocadas/atualizadas na geração mais recente



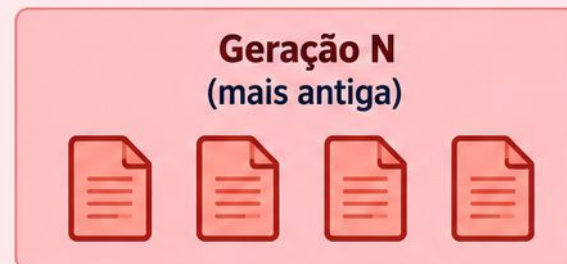
Quando uma página é acessada novamente, ela volta/mantém na geração mais recente (Geração 0).

3 Com o tempo, as gerações vão envelhecendo



4 Recuperação de memória

As páginas da geração mais antiga são as primeiras candidatas a serem removidas.



Página removida da memória

Referências bibliográficas

TANENBAUM, Andrew S.; BOS, Herbert.

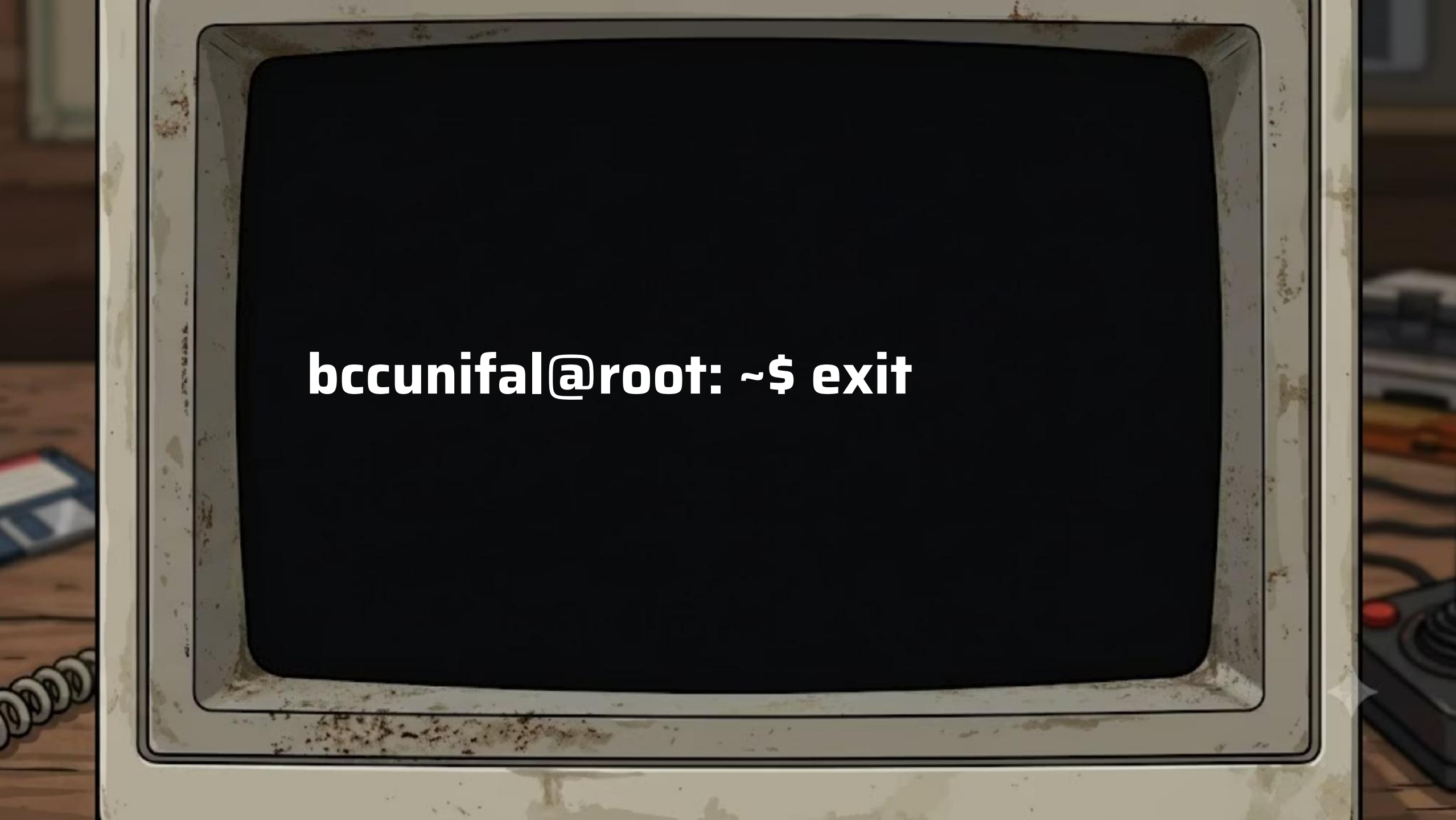
Sistemas Operacionais Modernos. 4. ed. São Paulo: Pearson, 2016.

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg.

Fundamentos de Sistemas Operacionais. 9. ed. Rio de Janeiro: LTC, 2015.

MAZIERO, Carlos A.

Sistemas Operacionais: Conceitos e Mecanismos. Curitiba: UFPR, 2019.

A terminal window is displayed on a computer monitor. The monitor has a light-colored, slightly textured bezel. The terminal window itself is a dark rectangle with a thin white border. Inside the terminal, the text 'bccunifal@root: ~\$ exit' is written in a white, monospaced font. The background of the entire image shows a desk with a keyboard and some papers, though they are out of focus.

bccunifal@root: ~\$ exit